E-COMMERCE PROGRESSIVE WEB APPLICATION (PWA) DEVELOPMENT

Problem Description

Traditional web applications often suffer from poor performance on mobile devices and become inaccessible during network disruptions. Furthermore, local merchants struggle to maintain consistent sales channels that accommodate users with varying internet reliability.

Motivation

The motivation is to engineer an accessible, fast, and reliable digital storefront utilizing Progressive Web App (PWA) technology. This ensures an app-like experience on mobile devices without the overhead of app store deployment, driving higher user engagement and conversion rates.

Project Objectives

Main Objective

- To develop an offline capable and functional E-commerce system with complete CRUD capabilities

Specific Objectives

- To implement a responsive, mobile-first frontend using PWA standards for offline resilience.
- To establish a secure, decoupled client-server architecture using a PHP-based REST API and MySQL.
- To implement rigorous security standards against modern web vulnerabilities.

System Requirements

Functional Requirements

- Users must be able to Create, Read, Update, and Delete (CRUD) their profiles and orders.
- Administrators must be able to manage the product inventory.
- The system must securely authenticate users and maintain active sessions.
- The application must process and validate all user inputs prior to database interaction.

Non-Functional Requirements

- **Mobile-First Design:** The UI must natively adapt to small screens before scaling to desktop environments.
- **Code Maintainability:** Clean, consistent, and well-commented code following industry best practices.
- **Performance:** The app must utilize service workers for asset caching to reduce load times.

System Architecture

Overall System Design:

The system utilizes a decoupled, modern architecture where the frontend (HTML/CSS/JS) operates independently from the backend (PHP). They communicate statelessly via JSON-formatted HTTP requests.

Client-Server Architecture Diagram (Conceptual):

[Client Browser / PWA (UI & Service Worker] <-- (JSON over HTTPS) --> [PHP REST API (Routing, Auth, Logic)] <-- (PDO) --> [MySQL Database]

Database Schema (ERD)

- **Users:** id (PK), email, password_hash, role, created_at
- **Products:** id (PK), name, description, price, stock
- **Orders:** id (PK), user_id (FK), total_amount, status

Authentication Mechanism

The system uses token-based authentication (or secure HttpOnly session cookies). Upon successful login, the backend verifies credentials and issues a secure session identifier, validating subsequent API requests.

Security Measures Used:

- SQL Injection prevention via PHP Data Objects (PDO) and prepared statements.
- Cross-Site Scripting (XSS) mitigation via strict output encoding (htmlspecialchars()).
- Cross-Site Request Forgery (CSRF) protection using synchronized anti-CSRF tokens for mutating requests.

Implementation Details

Technologies Used:

- **Frontend:** HTML5, CSS3, Vanilla JavaScript, Service Workers (PWA API).
- **Backend:** PHP 8.
- **Database:** MySQL.

Frontend Implementation

Built with a mobile-first philosophy using CSS Flexbox and Grid. JavaScript Fetch API is utilized for asynchronous DOM updates without page reloads. A manifest.json and sw.js (Service Worker) are implemented to allow the app to be installed on home screens and cache core assets.

Backend Implementation

A RESTful API architecture built in PHP. The backend strictly handles data validation, business logic, and database interactions, returning standardized HTTP status codes and JSON payloads.

Database Design

Relational tables structured to Third Normal Form (3NF) to minimize redundancy, utilizing foreign key constraints to ensure data integrity between users, products, and orders.

Challenges Faced

PWA cache invalidation

Managing PWA cache invalidation (ensuring users receive updated product data while still benefiting from offline capabilities) and structuring pure PHP into a clean API without the use of heavy frameworks.

Assumptions Made

It is assumed that the client browser supports modern ES6 JavaScript and Service Workers, and that the server environment supports PHP 8.x and MySQL.

Security Considerations

How authentication is handled

- Users authenticate via an API endpoint. Passwords are never stored in plain text.

How passwords are stored

- Passwords are hashed using the robust PASSWORD_BCRYPT algorithm with automatically generated salts before database insertion.

How data is protected

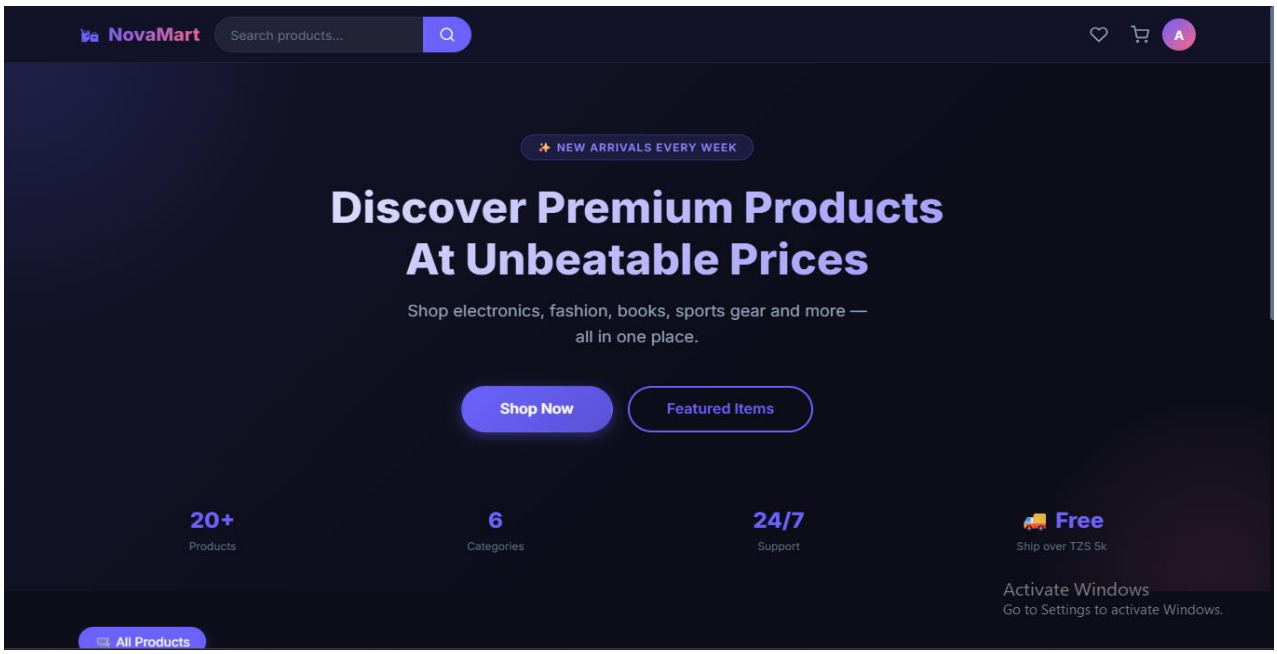
- Transport Layer Security (HTTPS) is mandated for all client-server communications to encrypt data in transit.

Access control implementation

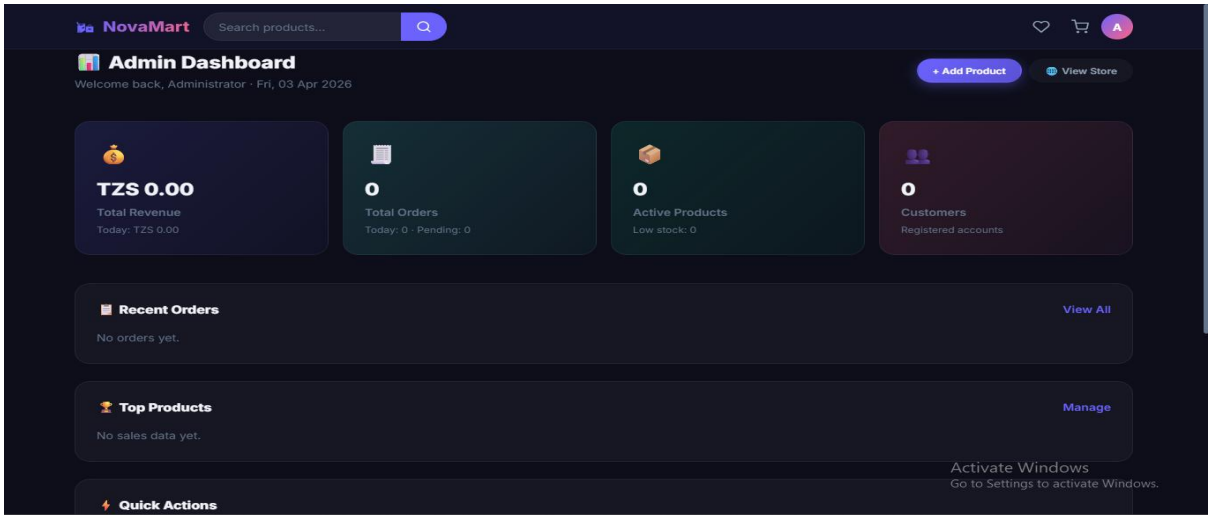
- Role-Based Access Control (RBAC) is enforced at the API level. The backend verifies the user's role (e.g., 'admin' vs 'customer') before allowing execution of sensitive CRUD operations like deleting a product.

Testing & Results

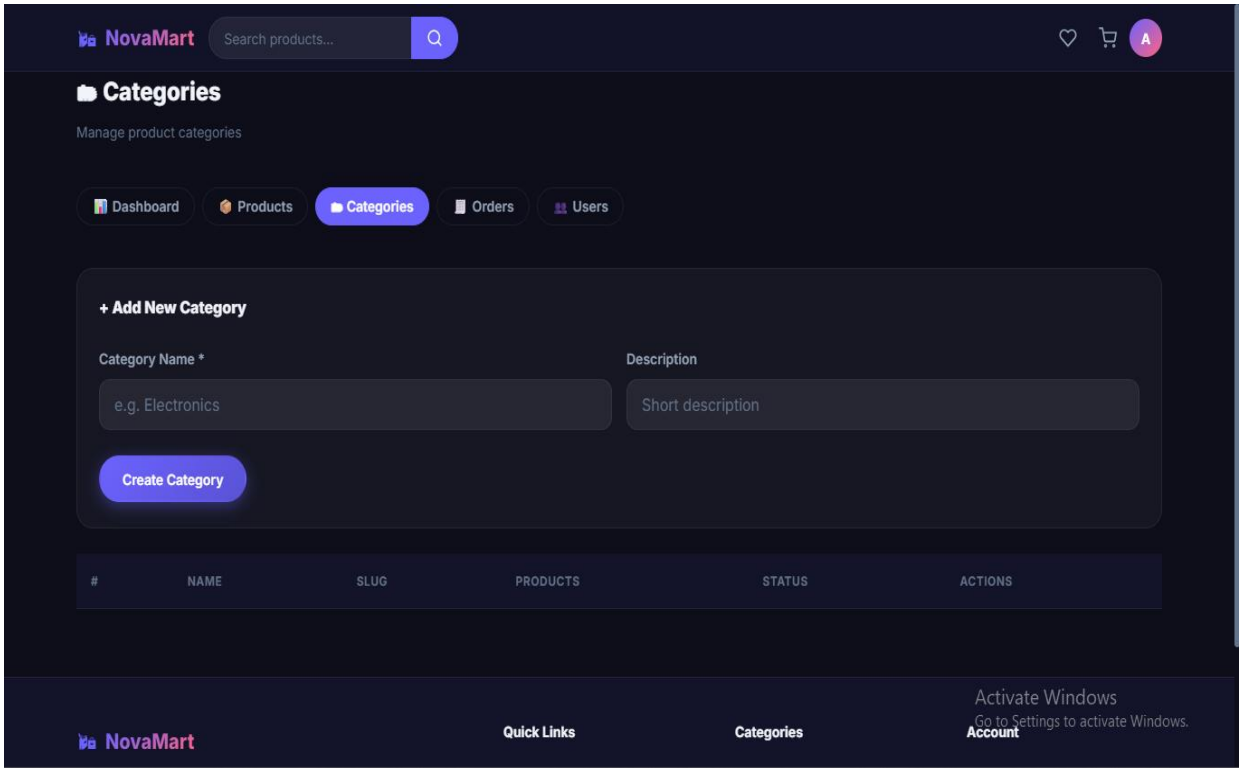
Screenshots of the application



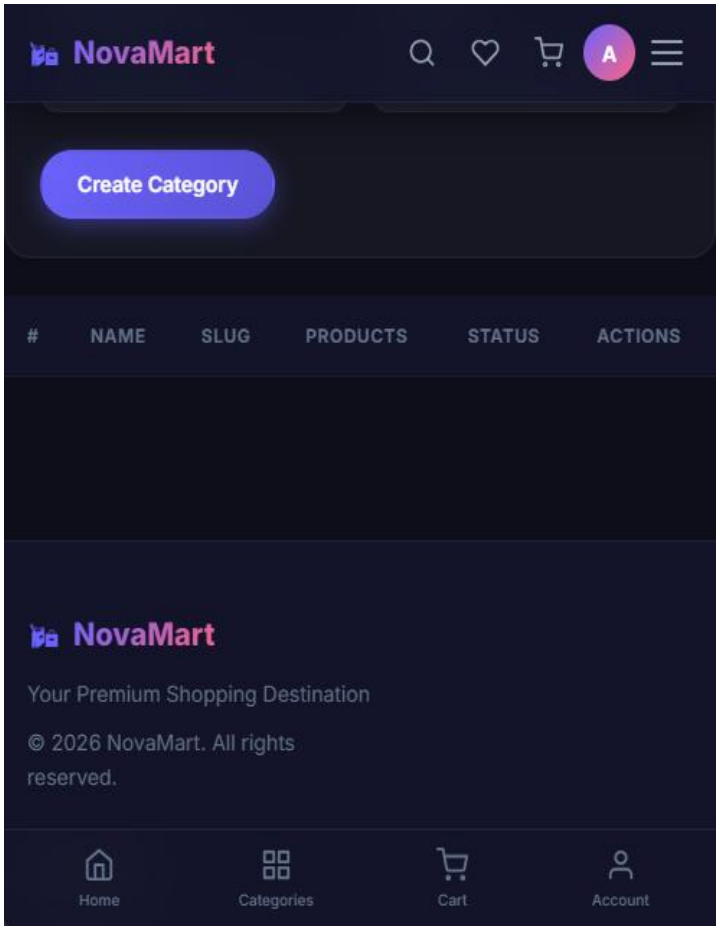
[Homepage Image]



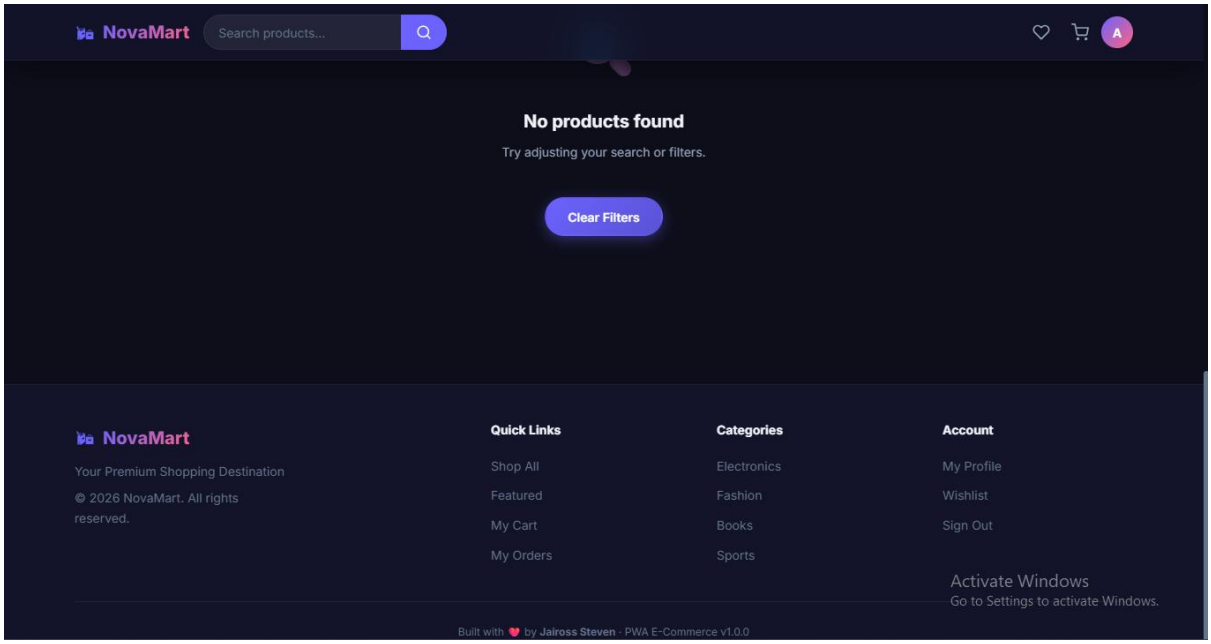
[Admin Dashboard]



[Category page]



[Mobile view]



[system footer]

Explanation of major features working

Registration, login, product browsing, and the PWA installation prompt have been successfully tested. The service worker correctly caches the application shell.

Description of incomplete features

Integration with a live payment gateway is currently simulated; real financial transactions are not yet processed.

Future Work

Possible Improvements:

- Integration of a secure, third-party payment gateway (e.g., Azam pay, Max-malipo, Stripe or PayPal).
- Implementation of Web Push Notifications to alert users of order status changes.
- Advanced product filtering and AI-driven recommendations.